

AI Assignment 4

Travelling Salesman Problem (TSP) Using Greedy Technique and Simulated Annealing

1 Assignment Overview

The **Travelling Salesman Problem (TSP)** is one of the most well-known optimization problems in computer science. A salesman must visit a number of cities exactly once and finally return to the starting city. The objective is to find a tour with the **minimum possible total travelling cost or distance**.

For this assignment, you will solve the same TSP instance using two different approaches:

1. **Greedy Technique** — to quickly construct a feasible solution.
2. **Simulated Annealing** — a metaheuristic optimization technique designed to improve solutions and escape local optima.

You will implement, test, compare, and analyze both techniques.

The purpose of this assignment is not only to obtain a short tour, but also to understand the differences between a **deterministic greedy heuristic** and a **probabilistic optimization algorithm**.

2 Problem Description

You are given N cities and the travelling cost between every pair of cities.

Your task is to find a tour that:

1. Starts from a specified starting city.
2. Visits every city exactly once.
3. Returns to the starting city.
4. Minimizes the total travelling cost.

For example, if the tour is:

$$A \rightarrow C \rightarrow D \rightarrow B \rightarrow A$$

then the total cost is:

$$Cost(A, C) + Cost(C, D) + Cost(D, B) + Cost(B, A)$$

The final return to the starting city **must be included** in the total cost.

3 Input Specification

The first line contains an integer:

$$N$$

where N is the number of cities.

The next N lines contain an $N \times N$ cost matrix.

The value at row i , column j represents the cost of travelling from city i to city j .

3.1 Assumptions

- $2 \leq N \leq 500$
- $Cost(i, i) = 0$
- All other costs are positive.
- The matrix may be symmetric or asymmetric.
- City numbering starts from 0.

For the main implementation, assume that every route starts from **City 0**.

4 Required Input Format

```
N
c[0][0] c[0][1] c[0][2] ... c[0][N-1]
c[1][0] c[1][1] c[1][2] ... c[1][N-1]
...
c[N-1][0] c[N-1][1] ... c[N-1][N-1]
```

5 Sample Input 1

```
5
0 10 15 20 25
10 0 35 25 17
15 35 0 30 28
20 25 30 0 22
25 17 28 22 0
```

One possible tour is:

```
0 -> 1 -> 4 -> 3 -> 2 -> 0
```

Its cost is:

$$10 + 17 + 22 + 30 + 15 = 94$$

6 Part A: Greedy Technique

6.1 Required Algorithm

Implement a greedy heuristic based on the **Nearest Neighbor Technique**.

Starting from City 0:

1. Mark the starting city as visited.
2. Among all unvisited cities, select the city with the minimum travelling cost from the current city.
3. Move to the selected city.
4. Mark it as visited.
5. Repeat until all cities have been visited.
6. Return to the starting city.
7. Calculate the total cost of the complete tour.

6.2 Example

Suppose the current city is City 2, and the unvisited cities have the following costs:

Destination	Cost
City 1	18
City 3	9
City 5	14

The greedy algorithm should choose **City 3**, because it has the minimum immediate cost.

6.3 Important Observation

The greedy choice produces a good next move based only on current information. However, the cheapest immediate move may lead to an expensive route later.

Therefore:

A greedy solution is not guaranteed to be globally optimal.

Your implementation must demonstrate this limitation through experimental comparison with Simulated Annealing.

7 Part B: Simulated Annealing

7.1 Background

Simulated Annealing is inspired by the physical process of annealing in metallurgy.

In physical annealing:

1. A material is heated to a high temperature.
2. Atoms move relatively freely.
3. The material is gradually cooled.
4. The atoms eventually settle into a low-energy state.

In optimization:

- A **solution** corresponds to a physical state.
- The **objective function value** corresponds to energy.
- A better solution has lower energy.
- The **temperature** controls the probability of accepting worse solutions.

The important advantage of Simulated Annealing is that it may occasionally accept a worse solution. This allows the algorithm to escape from a local optimum.

7.2 Solution Representation

Represent a tour as an array or list.

For example:

0 3 1 4 2

This represents:

```
0 -> 3 -> 1 -> 4 -> 2 -> 0
```

City 0 should remain fixed as the starting city.

7.3 Initial Solution

You must support at least one of the following methods:

Method 1: Random Initial Solution

Generate a random permutation of cities 1 through $N - 1$, while keeping City 0 fixed at the beginning.

Example:

```
0 -> 4 -> 1 -> 3 -> 2 -> 0
```

Method 2: Greedy Initial Solution

Use the tour generated by your Greedy algorithm as the initial solution for Simulated Annealing.

Recommended: Implement both methods and compare their performance.

8 Neighborhood Generation

Simulated Annealing requires a method for generating a neighboring solution.

You must implement the following neighborhood operation.

8.1 2-Opt / Segment Reversal

Select two positions i and j , where:

$$1 \leq i < j < N$$

Reverse the segment between those positions.

Example:

```
Current:  
0 -> 1 -> 2 -> 3 -> 4 -> 5 -> 0
```

After reversing the segment containing cities 2, 3, 4:

New:

0 -> 1 -> 4 -> 3 -> 2 -> 5 -> 0

This is a valid TSP tour because every city is still visited exactly once.

8.2 Optional Extension

You may additionally implement:

- Swap two cities.
- Insert one city into another position.
- Randomly choose among multiple neighborhood operators.

If additional operators are implemented, clearly explain them in your report.

9 Acceptance Rule

Let:

$$C_{\text{current}}$$

be the cost of the current solution and:

$$C_{\text{new}}$$

be the cost of the newly generated neighboring solution.

Define:

$$\Delta = C_{\text{new}} - C_{\text{current}}$$

Case 1: New Solution is Better

If:

$$\Delta \leq 0$$

accept the new solution.

Case 2: New Solution is Worse

If:

$$\Delta > 0$$

accept it with probability:

$$P = e^{-\Delta/T}$$

where:

- T = current temperature
- e = Euler's number

Generate a random value:

$$r \in [0, 1]$$

Accept the worse solution if:

$$r < P$$

10 Cooling Schedule

Use a geometric cooling schedule:

$$T_{\text{new}} = \alpha T_{\text{current}}$$

where:

$$0 < \alpha < 1$$

For example:

$$T_{\text{new}} = 0.995 T_{\text{current}}$$

Your program should allow the following parameters to be changed easily:

- Initial temperature T_0
- Cooling rate α
- Minimum temperature T_{min}

- Number of iterations per temperature level
- Maximum total number of iterations
- Random seed, if used

11 Recommended Parameter Values

You should experiment with different values.

A reasonable starting point is:

```
Initial Temperature: 1000
Cooling Rate: 0.995
Minimum Temperature: 0.001
Iterations per Temperature: 100
Maximum Iterations: 100000
```

These values are **not guaranteed to be optimal**.

Students are expected to conduct experiments and justify their final parameter choices.

12 Required Output

Your program must display results for both algorithms.

The output should include:

1. Algorithm name.
2. Final tour.
3. Total tour cost.
4. Execution time.

For Simulated Annealing, additionally display:

- Initial temperature.
- Cooling rate.
- Number of iterations.
- Initial solution cost.
- Best solution cost.
- Number of accepted moves.
- Number of worse solutions accepted.

13 Sample Output 1

For the sample input, the exact Simulated Annealing output may differ because the algorithm is randomized.

A possible output format is:

```
=====
TRAVELLING SALESMAN PROBLEM
=====

Number of Cities: 5

----- GREEDY METHOD -----
Tour:
0 -> 1 -> 4 -> 3 -> 2 -> 0

Total Cost: 94
Execution Time: 0.0002 seconds

----- SIMULATED ANNEALING -----
Initial Solution:
0 -> 3 -> 4 -> 2 -> 1 -> 0

Initial Cost: 115

Best Tour Found:
0 -> 1 -> 4 -> 3 -> 2 -> 0

Best Cost: 94

Initial Temperature: 1000
Cooling Rate: 0.995
Total Iterations: 100000

Accepted Moves: 28341
Worse Moves Accepted: 13785

Execution Time: 0.0824 seconds

----- COMPARISON -----
Greedy Cost: 94
Simulated Annealing Cost: 94

Best Method for This Run:
Both methods produced the same cost.
=====
```

Note: The numbers related to iterations, accepted moves, execution time, and Simulated Annealing results may vary depending on the implementation and random initialization.

14 Sample Input 2

```
6
0 12 10 19 8 14
12 0 3 7 2 11
10 3 0 6 20 9
19 7 6 0 4 5
8 2 20 4 0 13
14 11 9 5 13 0
```

15 Possible Sample Output 2

```
=====
Number of Cities: 6
=====

GREEDY METHOD
Tour:
0 -> 4 -> 1 -> 2 -> 3 -> 5 -> 0

Total Cost: 38

SIMULATED ANNEALING
Best Tour:
0 -> 4 -> 1 -> 2 -> 3 -> 5 -> 0

Best Cost: 38

Initial Cost: 59
Total Iterations: 100000
Accepted Moves: 21457
Worse Moves Accepted: 9872

COMPARISON
Greedy Cost: 38
Simulated Annealing Cost: 38
=====
```

The exact result may differ between executions.

16 Experimental Requirements

You must test your implementation on **at least five different TSP instances**.

Recommended instance sizes:

Test Case	Number of Cities
Small	10
Small-Medium	20
Medium	50
Large	100
Larger	200 or more

You may use:

- Self-generated cost matrices.
- Randomly generated Euclidean coordinates.
- Standard TSP benchmark instances.

If external benchmark datasets are used, the source must be properly cited in your report.

17 Required Experiments

17.1 Experiment 1: Greedy vs. Simulated Annealing

For each test instance, record:

- Greedy cost.
- Best Simulated Annealing cost.
- Percentage improvement.
- Greedy execution time.
- Simulated Annealing execution time.

Use:

$$\text{Improvement Percentage} = \frac{\text{Greedy Cost} - \text{SA Cost}}{\text{Greedy Cost}} \times 100$$

17.2 Experiment 2: Different Initial Temperatures

Test at least three different values of T_0 .

For example:

```
T0 = 100
T0 = 1000
T0 = 5000
```

Discuss how the initial temperature affects:

- Exploration.
- Acceptance of worse solutions.
- Final solution quality.
- Execution time.

17.3 Experiment 3: Different Cooling Rates

Test at least three cooling rates.

For example:

0.90
0.95
0.995

Discuss:

- Fast cooling.
- Slow cooling.
- Solution quality.
- Computational cost.

17.4 Experiment 4: Random vs. Greedy Initialization

Run Simulated Annealing using:

1. A random initial tour.
2. The greedy tour as the initial solution.

Perform each experiment at least **five times** because Simulated Annealing is randomized.

Compare:

- Best result.
- Average result.
- Worst result.
- Average execution time.

18 Experimental Results Table

Your report should contain a table similar to the following:

Instance	Cities	Greedy	SA Best	SA Avg.	Improve.	G Time	SA Time
Test 1	10						
Test 2	20						
Test 3	50						
Test 4	100						
Test 5	200						

19 Implementation Requirements

Your implementation must contain separate functions or modules for at least the following:

```
readInput()
calculateTourCost()
greedyTSP()
generateRandomTour()
generateNeighbor()
simulatedAnnealing()
printResults()
```

You may add additional helper functions as necessary.

20 Correctness Requirements

Your submitted program must ensure that every generated solution:

- Starts at the designated starting city.
- Visits every city exactly once.
- Returns to the starting city.
- Contains no duplicate city in the intermediate tour.
- Contains all cities exactly once before returning.

You should implement a function to validate a tour during development and testing.

21 Complexity Analysis

Your report must include a complexity analysis.

21.1 Greedy Technique

Discuss the approximate time complexity of the Nearest Neighbor implementation.

A typical implementation examines unvisited cities at each step, resulting in approximately:

$$O(N^2)$$

time complexity.

21.2 Simulated Annealing

Let I be the total number of iterations.

Discuss the complexity based on:

- Neighbor generation.
- Tour cost calculation.
- Number of iterations.

A straightforward implementation that recalculates the entire tour cost during every iteration may require approximately:

$$O(I \times N)$$

time.

You should explain whether and how your implementation could improve this.

22 Deadline and Submission Instructions

Submission Deadline: 7 September 2026, 11:59 PM

Submit the complete assignment as a single ZIP archive. The archive must include all source-code files, input files, output or result files, the report, and any other files required to compile and run the program.

Name the ZIP archive using the following format:

2205ddd.zip

Here, ddd represents the last three digits of your student ID. For example, if your student ID ends in 123, the archive must be named 2205123.zip.

Before submission, verify that the archive opens correctly and contains all required files. Submissions with an incorrect filename, missing files, or files that cannot be compiled or executed may be penalized.

23 Marks Distribution

Component	Marks
Correct TSP representation and input handling	10
Greedy algorithm implementation	15
Simulated Annealing implementation	25
Neighborhood generation and acceptance rule	10
Parameter configuration and cooling schedule	10
Experimental evaluation	10
Comparison and complexity analysis	10
Code quality and documentation	5
Report quality	5
Total	100